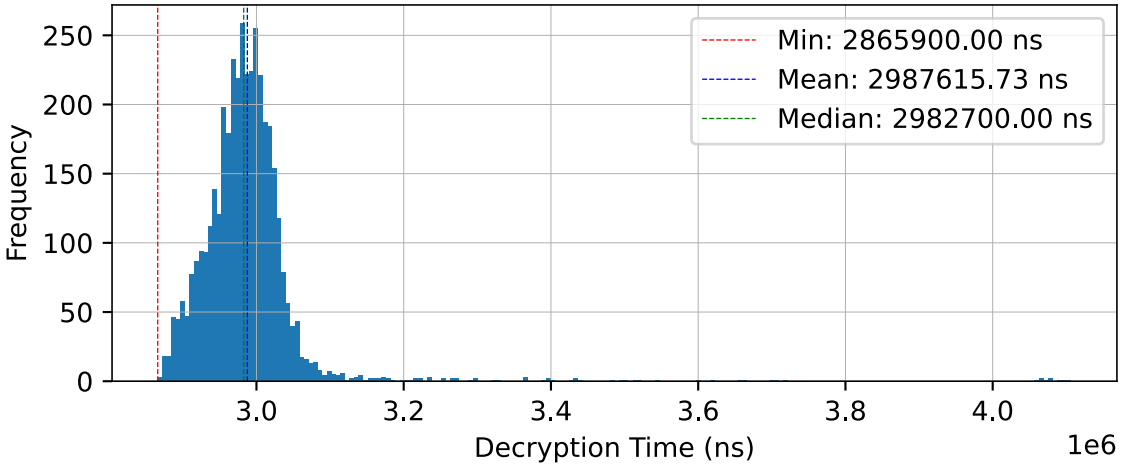
University of Geneva

Faculty of Science

Timing Attacks

Advanced Security
14X040



Michel Jean Joseph Donnet

2026-05-15

Contents

1 Introduction 3
2 RSA algorithm 4
2.1 Key generation 4
2.2 Encryption and decryption 4
3 Variance-based timing attack 6
3.1 Square-and-multiply algorithm 6
3.2 Exploiting timing variations 7
3.3 Mathematical analysis 8
3.3.1 Probability of a correct guess 8
3.3.2 Variance analysis 8
3.3.3 Impact of previous errors on the variance 9
3.4 Code implementation 9
3.4.1 Sources of noise 10
3.4.1.1 CPU cache effects 10
3.4.1.2 Branch prediction 10
3.4.1.3 Garbage collection 10
3.4.1.4 Multiplication optimizations 11
3.4.1.5 Other optimizations 11
3.4.2 Reducing the impact of noise 11
3.4.2.1 Reducing the impact of the garbage collector 11
3.4.2.2 Averaging timing measurements 12
3.4.2.3 Warm-up and optimizations of parameters 13
3.4.3 Performance of the attack 15
4 Pearson-based timing attack 18
4.1 Pearson correlation coefficient 18
4.2 Designing the model of expected timings 18
4.3 Advantages of the Pearson-based approach 19
4.4 Disadvantages of the Pearson-based approach 20
4.5 Results of the Pearson-based approach 20
4.6 Why the timing attack on RSA is still relevant 22
5 Fermat's factorization method on RSA 23
5.1 Security of RSA and factoring large integers 23
5.2 Description of Fermat's factorization method 23
5.3 From theory to the algorithm 23
5.4 Results of Fermat's factorization method on RSA 24
6 Conclusion 26
6.1 Usage of AI 26
6.1.1 ChatGPT (Free) 26
6.1.2 Claude Sonnet 4.6 (Free) 26
6.1.3 Github Copilot (Student Plan) 27
6.1.4 DeepL (Free) 27
Bibliography 28

1 Introduction

The cryptography goal is to ensure the confidentiality, integrity, and authenticity of information, and it has become increasingly important in the digital age with the rise of the internet and digital communication where sensitive information is frequently transmitted.

To achieve these goals, various cryptographic algorithms and protocols have been developed, including symmetric and asymmetric encryption, hashing, and digital signatures. These algorithms are based on mathematical problems that are computationally difficult to solve, such as factoring large integers or computing discrete logarithms, which provide the basis for their security since they are believed to be infeasible to break with current computational resources.

However, although these algorithms are designed to be secure against direct attacks, their implementation can introduce vulnerabilities that can be exploited by attackers. The vulnerabilities reside in the way the algorithms are implemented and executed, rather than in the algorithms themselves. Indeed, attackers can exploit side channels, which are unintended information leaks that occur during the execution of cryptographic algorithms such as timing information, power consumption, electromagnetic emissions, or even sound. By analyzing these side channels, attackers can gain insights into the internal workings of the cryptographic algorithm and potentially recover sensitive information such as secret keys.

In this report, we will focus on the timing attack on RSA, which is a type of side-channel attack that exploits the timing information of the decryption operation to recover the private key.

We will also discuss about the security of RSA based on its key generation process, which involves selecting two large prime numbers and computing their product. The Fermat factorization method is a technique for factoring large integers that can be used to break RSA encryption if the modulus n is not chosen properly.

2 RSA algorithm

RSA is a widely used asymmetric encryption algorithm that relies on the difficulty of factoring large integers to provide security. It allows in particular for sharing a symmetric key securely over an insecure channel, enabling secure communication between parties without the need for a pre-shared secret key.

The base principle of RSA resides in the use of a pair of keys: a public key for encryption and a private key for decryption. The RSA algorithm consists of three main steps: key generation, encryption, and decryption.

2.1 Key generation

The key generation process involves selecting two large prime numbers, p and q, and computing their product $n = p \times q$, which serves as the modulus for both the public and private keys. The choice of p and q is crucial for the security of RSA, as the difficulty of factoring n into its prime factors is what provides the security of the algorithm. Indeed, if an attacker can factor n into p and q, they can compute the private key and break the encryption.

The algorithm of key generation can be summarized as follows:

1. Choose two distinct large prime numbers p and q.
2. Compute $n = p \times q$ and $\varphi(n) = (p-1)(q-1)$.
3. Choose an integer e such that
   - $1 < e < \varphi(n)$
   - $\gcd(e, \varphi(n)) = 1$
4. Compute d such that $e \cdot d \equiv 1 \bmod \varphi(n)$.

The public key consists of the pair (n, e), while the private key consists of the pair (n, d).

2.2 Encryption and decryption

The encryption process takes a plaintext message m and computes the ciphertext

$$c = m^e \bmod n$$

using the public key, and the decryption process takes the ciphertext c and computes the plaintext message

$$m = c^d \bmod n$$

using the private key.

By construction:

$$\begin{aligned} c^d \bmod n &= m^{ed} \bmod n \\ &= m^{1+k\times\varphi(n)} \bmod n \quad \text{since } ed \equiv 1 \bmod \varphi(n) \\ &= m \times \left(m^{\varphi(n)}\right)^k \bmod n \\ &= m \times 1^k \bmod n \quad \text{by Euler's theorem} \\ &= m \bmod n \end{aligned}$$

So the decryption process correctly recovers the original plaintext message m.

For convenience, we will use the following notations throughout the report:

- n: the modulus, which is the product of two large prime numbers p and q.
- e: the public exponent, which is an integer that is coprime to $\varphi(n)$.
- d: the private exponent, which is an integer such that $e \cdot d \equiv 1 \bmod \varphi(n)$.
- m: the plaintext message, which is an integer that is less than n.
- c: the ciphertext, which is an integer that is less than n.

3 Variance-based timing attack

Before the exchange of symmetric keys, the client and the server need to perform an RSA encryption and decryption operation to establish a secure communication channel. The timing of these operations can be exploited by attackers to infer information about the private key.

As mentioned earlier, the security of RSA relies on the difficulty of factoring large integers, but the implementation of RSA can introduce vulnerabilities that can be exploited by attackers.

The goal of the attacker is to recover the unknown private key d thanks to the knowledge of the public key (n, e) and the ability to perform decryption operations on the server using the RSA algorithm.

3.1 Square-and-multiply algorithm

Timing attacks on RSA exploit the fact that the time it takes to perform certain operations in the RSA algorithm can vary based on the input values and the internal state of the algorithm.

Indeed, depending on the algorithm used for the modular exponentiation operation, the execution time can vary based on the value of the exponent and the input ciphertext, as explained by Paul C. Kocher in his original paper "Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems" [1].

For instance, the square-and-multiply algorithm, which is commonly used for modular exponentiation, can have different execution times based on the bits of the exponent.

```python
def square_and_multiply(y, x, n):
    s = 1
    y %= n

    while x > 0:
        if (x % 2) == 1:
            s = (s * y) % n  # Extra multiplication here when x is odd !
        y = (y * y) % n
        x = x >> 1

    return s
```

As computing directly $y^x \bmod n$ is computationally expensive and requires a large amount of memory, the square-and-multiply algorithm is used to compute $y^x \bmod n$ efficiently by breaking down the exponentiation into a series of squarings and multiplications based on the binary representation of the exponent x. Indeed, the square-and-multiply algorithm computes a stack of $y^{2^i} \bmod n$ for $i = 0, 1, 2, ..., \lfloor \log_2(x) \rfloor$, decomposes x into its binary representation, and iteratively computes $y^x \bmod n$ by taking the corresponding powers.

For instance, suppose we want to compute $6^{13} \bmod 17$.

- The binary representation of 13 is 1101, which corresponds to the powers of 2: $2^3 + 2^2 + 2^0$.
- We start with $s = 1$ and $y = 6$.
- For the first bit (1), we compute
  - $s = (s \cdot y) \bmod 17 = (1 \cdot 6) \bmod 17 = 6$
  - $y = y^2 \bmod 17 = 6^2 \bmod 17 = 2$.
- For the second bit (0), we compute
  - s remains unchanged since the bit is 0, so $s = 6$.
  - $y = y^2 \bmod 17 = 2^2 \bmod 17 = 4$. Note that here, y^2 corresponds to $y^{2^2} = y^{2 \cdot 2} = y^4$.
- For the third bit (1), we compute
  - $s = (s \cdot y) \bmod 17 = (6 \cdot 4) \bmod 17 = 24 \bmod 17 = 7$
  - $y = y^2 \bmod 17 = 4^2 \bmod 17 = 16$.
- For the fourth bit (1), we compute
  - $s = (s \cdot y) \bmod 17 = (7 \cdot 16) \bmod 17 = 112 \bmod 17 = 15$
  - $y = y^2 \bmod 17 = 16^2 \bmod 17 = 256 \bmod 17 = 1$.

  The obtained result is $s = 15$, which corresponds to $6^{13} \bmod 17$.

The efficiency of the algorithm comes from the fact that it iteratively computes $y^x \bmod n$ by performing small multiplication and squaring operations thanks to the modular properties instead of performing a single large exponentiation operation.

But the timing of this algorithm can vary based on the value of the exponent x and the input y. Indeed, as seen in the above example, the algorithm performs an extra multiplication only when the bit of x is 1, which can lead to a longer execution time compared to when the bit is 0. This variation in timing is the vulnerability that timing attacks exploit.

3.2 Exploiting timing variations

Attackers can exploit the timing variations in the square-and-multiply algorithm to recover the private key d by measuring the time it takes for the server to perform decryption operations. Note that the attacker must know the modulus n, which is public.

First, the attacker sends multiple random ciphertexts c to the server and measures the time it takes for the server to perform the decryption operation $c^d \bmod n$.

Then for each bit of the private key d, the attacker guesses the value of the bit (0 or 1) and computes the expected timing of the decryption operation based on the guess for all the ciphertexts c.

Finally, the attacker subtracts his obtained timing measurements from the timing measurements obtained from the server and analyzes the variance of the obtained results. If the variance is low, it means that the guess for the bit is correct whereas if the variance is high, it means that the guess for the bit is incorrect.

By repeating this process for all the bits of the private key d, the attacker can recover the entire private key.

3.3 Mathematical analysis

3.3.1 Probability of a correct guess

We consider N random ciphertexts $c_1, c_2, ..., c_N$ that the attacker sends to the server for decryption.

Let T_i be the time taken for the server to perform the decryption operation $c_i^d \bmod n$ for a given ciphertext c_i.

Let x_b be the guess for the b first bits of the private key d.

Let $t(c_i, x_b)$ be the time taken to perform the decryption operation according to the guess x_b for the b first bits of d.

If the guess x_b is correct, the timing error $T_i - t(c_i, x_b)$ will be small, indicating a good match between the observed and expected timings. We define F as the probability to observe the timing error $T_i - t(c_i, x_b)$ if the guess x_b is correct. As each timing measurement is independent, the probability of a correct guess for the b first bits of d is proportional to the product of the probabilities of F for all the ciphertexts c_i:

$$P(x_b) \propto \prod_{i=1}^{N} F(T_i - t(c_i, x_b) \mid x_b)$$

Suppose that the $b-1$ bits of d are correct, and we want to guess the value of the b-th bit.

The probability of having $x_b = 1$ is:

$$P(x_b = 1) = \frac{\prod_{i=1}^{N} F(T_i - t(c_i, x_b) \mid x_b = 1)}{\prod_{i=1}^{N} F(T_i - t(c_i, x_b) \mid x_b = 0) + \prod_{i=1}^{N} F(T_i - t(c_i, x_b) \mid x_b = 1)}$$

If $P(x_b = 1) > 0.5$, it means that the guess $x_b = 1$ is more likely to be correct than the guess $x_b = 0$, and thus we can conclude that the b-th bit of d is 1.

The problem is that computing the distribution of F is not feasible.

3.3.2 Variance analysis

Instead of computing the exact distribution of F, we can analyze the variance of the timing errors for the two guesses $x_b = 0$ and $x_b = 1$.

Suppose the private key d has l bits. The time taken for the decryption operation can be denoted as $T_i = \sum_{k=1}^{l} t_k + \varepsilon$ with t_k the time taken for the k-th step of the algorithm and ε representing the timing noise caused by various factors such as system load, network latency, or other sources of noise.

Suppose that x_{b-1} is known to be correct. The attacker can compute the time taken for each step of the first $b-1$ bits as $\sum_{k=1}^{b-1} t_k$.

The time difference is then:

$$T_i - t(c_i, x_{b-1}) = \sum_{k}^{l} t_k + \varepsilon - \sum_{k}^{b-1} t_k = \sum_{k=b}^{l} t_k + \varepsilon$$

If the guess x_b is correct, the time difference will be:

$$T_i - t(c_i, x_b) = \sum_{k=b+1}^{l} t_k + \varepsilon$$

And if the guess x_b is incorrect, the time difference will be:

$$T_i - t(c_i, x_b) = \sum_{k}^{l} t_k + \varepsilon - \left(\sum_{k}^{b-1} t_k + t_{b_{\text{incorrect}}} \right) = \sum_{k=b+1}^{l} t_k + \varepsilon + \left(t_{b_{\text{correct}}} - t_{b_{\text{incorrect}}} \right)$$

.

We can constate that the time difference for the correct guess x_b is smaller than the time difference for the incorrect guess x_b by a factor of $t_{b_{\text{correct}}} - t_{b_{\text{incorrect}}}$, meaning that the variance of the time differences for the correct guess x_b will be smaller than the variance of the time differences for the incorrect guess x_b.

So by analyzing the variance of the time differences for the two guesses $x_b = 0$ and $x_b = 1$, the attacker can determine which guess is more likely to be correct, and thus recover the bits of the private key d one by one.

3.3.3 Impact of previous errors on the variance

Now, suppose that the c-th bit of d is incorrect for some $c < b - 1$. The time difference for the correct guess x_b will be:

$$T_i - t(c_i, x_b) = \sum_{k}^{l} t_k + \varepsilon - \left(\sum_{k}^{c-1} t_k + \sum_{k=c}^{b} t_k \right) = \sum_{k=b+1}^{l} t_k + \varepsilon + \left(\sum_{k=c}^{b} \left(t_{k_{\text{correct}}} - t_{k_{\text{incorrect}}} \right) \right)$$

We have t_k which starts to be different for the correct and incorrect guesses for all the bits from c to b since the intermediate steps of the algorithm will be executed differently due to the error in the guess for the c-th bit, leading to a different t_k for all the bits from c to b even if the guess for the b-th bit is correct.

So when the guess x_b has some previous errors, the variance of the correct guess will start to be similar to the variance of the incorrect guess due to the error propagating through the intermediate steps of the algorithm.

This property can be exploited to detect errors in the guess for the b-th bit. Indeed, attacker can keep track of the variance of the time differences and come back to a previous guess if the variance starts to be too similar for both the correct and incorrect guesses, indicating that there might be an error in the previous bits of the guess.

3.4 Code implementation

The code implementation was done first in Python, and then in Rust and C++ to try to reduce the noise in the timing measurements. Finally, thanks to a master's student's knowledge of a Python library, it was possible to perform precise timing measurements in Python. I therefore decided to implement the attack in Python using this library, which made it easier to implement, understand, and demonstrate.

I used AI[1] and my own knowledge to implement and debug the code, and I also referred to the blog post "Kocher's Timing Attack: A Journey from Theory to Practice" [2], which

[1] ChatGPT didn't want to implement the attack due to the ethical implications of providing code for a timing attack, but Claude Sonnet was able to provide a code implementation for the attack, which I used as a starting point for my implementation.

provided a demonstration of realistic timing attacks on RSA and some techniques to mitigate the impact of noise in the timing measurements.

However, the implementation described in the blog post didn't work for me, so I had to implement the attack myself. In doing so, I realized just how difficult it was to achieve good results due to the noise in the timing measurements, which turned out to be a very rewarding experience.

3.4.1 Sources of noise

There are several sources of noise that can affect the timing measurements in programming languages, especially in high-level languages like Python.

3.4.1.1 CPU cache effects

The CPU cache is a small, fast memory that is used to store recently accessed data and instructions to speed up access to them. There are several levels of CPU cache (L1, L2, L3) which differ in size and speed, with L1 being the smallest and fastest, and L3 being the largest and slowest. When the CPU needs to access data or instructions, it first checks if they are in the cache. If they are, it can access them quickly, but if they are not, it has to fetch them from the main memory, which is much slower.

This can lead to significant variability in the timing measurements since the cache state can change based on the input values and the internal state of the algorithm, leading to different cache hits and misses and thus different timing measurements.

3.4.1.2 Branch prediction

Branch prediction is a technique used by modern CPUs to improve performance by guessing the outcome of conditional statements and executing instructions based on those guesses. When the CPU encounters a conditional statement (e.g., an if statement), it makes a guess about which branch of the code will be executed next based on past behavior and patterns. This allows the CPU to continue executing instructions without waiting for the outcome of the conditional statement.

So if the CPU correctly predicts the branch, it can continue executing instructions without interruption. But if the CPU incorrectly predicts the branch, it has to discard the incorrectly executed instructions and fetch the correct instructions, leading to a significant delay compared to the case where the branch is correctly predicted.

That's why branch prediction can cause variability in the timing measurements as different inputs can lead to different execution paths and thus different branch predictions.

3.4.1.3 Garbage collection

Garbage collection is a form of automatic memory management that is used in many programming languages, including Python. It is responsible for automatically freeing up memory that is no longer in use by the program, which can help prevent memory leaks and improve performance.

However, the garbage collector can be triggered at any time during the execution of the program. When the garbage collector runs, it can cause significant delays in the execution

of the program: it has to pause the execution of the program, scan the memory for objects that are no longer in use, and free up the memory occupied by those objects.

This can lead to significant variability in the timing measurements since the garbage collector can be triggered at different times, resulting in different timing measurements for the same operations.

3.4.1.4 Multiplication optimizations

The usage of modern programming languages introduces various optimizations that can affect the timing measurements, such as the optimization of multiplication operations.

Indeed, modern programming languages often use optimized algorithms for multiplication such as Karatsuba, Toom-Cook or Schönhage-Strassen, especially for large integers multiplication, which can significantly reduce the time taken for multiplication operations compared to the traditional algorithm. The choice of the multiplication algorithm depends on the size of the integers being multiplied.

As each algorithm has different performance characteristics, the timing measurements vary based on the size of the integers being multiplied.

3.4.1.5 Other optimizations

There are also other optimizations that can be introduced by the programming language or the compiler, such as loop unrolling, instruction reordering, or just-in-time compilation, which can further introduce variability in the timing measurements.

So the sources of noise in timing measurements can be quite significant, especially in high-level programming languages like Python, and they can totally mask the timing variations caused by the square-and-multiply algorithm.

3.4.2 Reducing the impact of noise

There are several techniques that can be used to try to reduce the impact of these sources of noise in timing measurements.

Inspired by the blog post "Kocher's Timing Attack: A Journey from Theory to Practice" [2], I try to implement some of them, trying to mitigate the impact of noise and thus increase the chances of success of the attack.

3.4.2.1 Reducing the impact of the garbage collector

As mentioned in Section 3.4.1, the garbage collector is a significant source of noise in the timing measurements, so one way to reduce the noise is to disable the garbage collector during the timing measurements.

This can be done manually using the `gc` module in Python, which provides functions to enable and disable the garbage collector.

However the right approach to perform the precise timing measurements was not to manually disable the garbage collector, but rather to use the `timeit` module in Python.

Indeed, the `timeit` module, which is a part of the Python standard library, is specifically designed for measuring the execution time of small code snippets with high precision. It

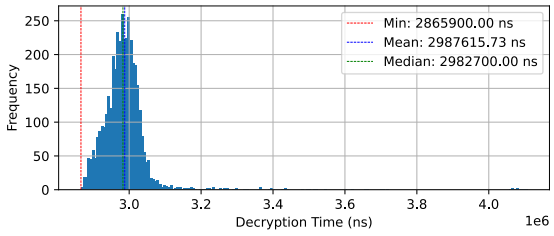
automatically handles various sources of noise, including disabling the garbage collector during the timing measurements.

The Figure 1 shows the distribution of the timing measurements obtained with the manual management of the garbage collector and with the `timeit` module, which handles GC automatically, for a fixed ciphertext and a fixed private key.

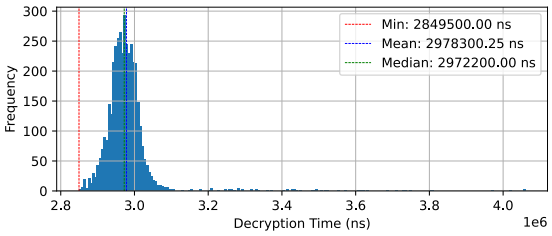
In case of small key (Figure 1c and Figure 1d), the timing measurements obtained with the `timeit` module are less noisy since the frequency of the timing measurements is around 1.5 times higher than with the manual management of the garbage collector, which means that the timing measurements obtained with the `timeit` module are more stable and less affected by noise compared to the timing measurements obtained with manual management of the garbage collector.

However, in case of big key (Figure 1a and Figure 1b), the timing measurements obtained with the `timeit` module are equivalent to the timing measurements obtained with the manual management of the garbage collector, which means that the two approaches are equivalent in this case, and the noise in the timing measurements is not significantly reduced by using the `timeit` module compared to the manual management of the garbage collector.

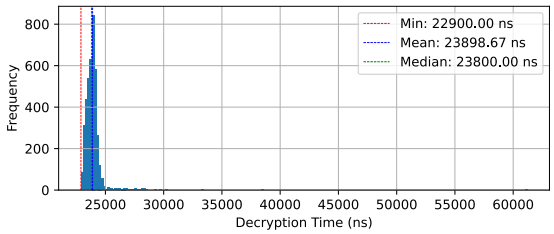
For the attack implementation, I used the `timeit` module for the timing measurements instead of manually disabling the garbage collector to simplify the process.



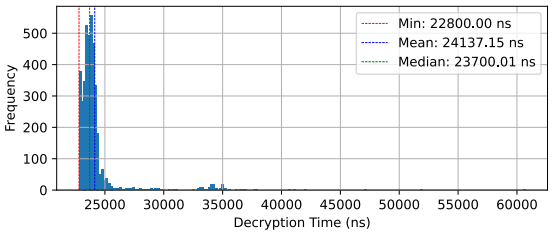
(a) Manual GC disable with big key



(b) Using timeit with big key



(c) Manual GC disable with small key



(d) Using timeit with small key

Figure 1: Comparison of timing measurements with manual garbage collector (GC) disable versus using timeit (GC handled automatically)

3.4.2.2 Averaging timing measurements

Once the garbage collector is disabled, there are still other sources of noise that can affect the timing measurements, as described in Section 3.4.1.

The most common approach to try to extract the signal from the noise in timing measurements is to perform a large number of timing measurements and then average them. In this way, we can reduce the impact of random fluctuations in the timing measurements.

However, as shown on Figure 1, there are some samples for which the timing measurements are significantly higher than the others due to the sources of noise described in Section 3.4.1, which can skew the average and thus make it not representative of the true decryption time. On top of that, for large key sizes, the timing measurements have a much higher variance, which can make the average not representative of the true decryption time even if there are no outliers in the timing measurements.

Using the mean was my first approach, but the mean is very sensitive to outliers, and thus can be significantly skewed by the presence of outliers in the timing measurements, leading to a wrong conclusion about the guess for the bit of the private key. In fact, as shown in Figure 1, the mean tends to be shifted to the right relative to the peak frequency due to the presence of outliers, as is the case, for example, in Figure 1d.

Then I tried to use the median, which is less sensitive to outliers compared to the mean, and thus can be more representative of the true decryption time. It works better than the mean for small key sizes, but for large key sizes, the variance of the timing measurements is so high that even the median is not sufficient to extract the signal from the noise and thus distinguish between the two cases.

Finally, I realized that the minimum timing measurement is a very good estimator of the true decryption time, as it is the one that is least affected by the noise. In fact, the noise in the timing measurements is an additive noise, meaning that it can only add latency to the timing measurements, but it can never subtract latency from the timing measurements.

The impact of the choice of the estimator (mean, median, minimum) on the attack implementation was huge, as it allowed to significantly decrease the error rate of the attack.

3.4.2.3 Warm-up and optimizations of parameters

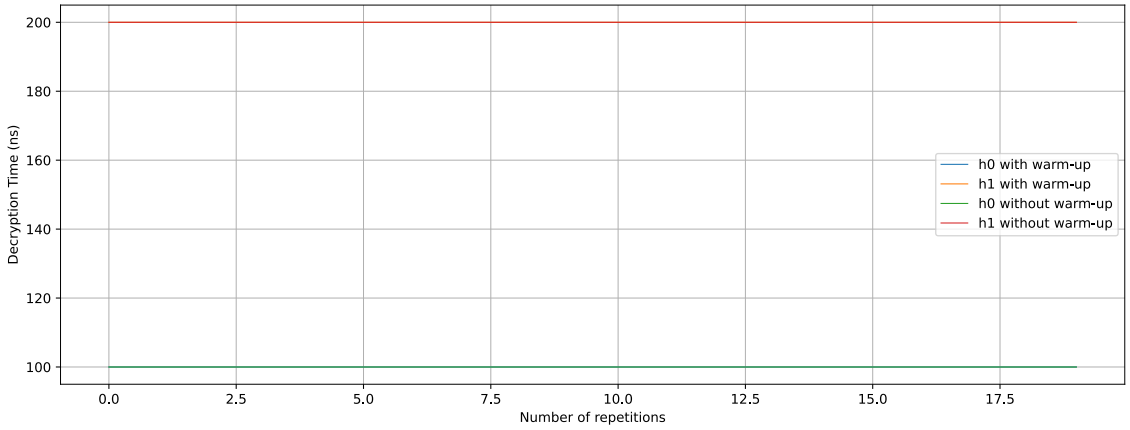
To reduce the impact of noise caused by the CPU cache effects and branch prediction, a warm-up phase was added before the timing measurements.

The warm-up phase consists in performing a certain number of decryption operations before the timing measurements. This allows the CPU to load the relevant data and instructions into the cache and the branch predictor to learn the execution patterns of the algorithm, which can help to reduce the impact of noise caused by these factors and thus increase the chances of success of the attack.

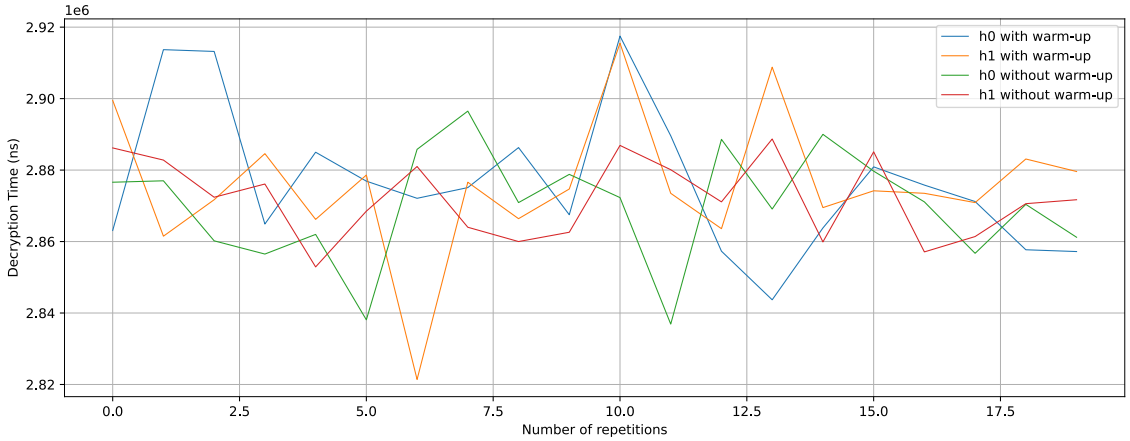
However, by looking at the results with and without the warm-up phase in Figure 2, we can see that the warm-up phase does not have a significant impact on the timing measurements since it does not allow to better distinguish between the two cases with and without the extra multiplication.

In fact, adding a warm-up phase will not allow to better find the minimum timing measurement, especially for large key sizes where the variance of the timing measurements is very high, and thus it will not help to reduce the impact of noise caused by the CPU cache effects and branch prediction.

In our case, it therefore makes more sense to increase the number of time measurements to improve the chances of finding a good minimum value, rather than adding a warm-up phase to stabilize measurements that remain noisy.



(a) Small key size



(b) Big key size

Figure 2: Comparison of differences with (h1) and without (h0) extra multiplications with and without warm-up phase with minimum timing measurements for small and big key sizes

The Figure 2b shows that for a big key size, the timing measurements for the case with the extra multiplication (h1) and the case without the extra multiplication (h0) are very close to each other, making it difficult to distinguish between the two cases based on the timing measurements.

So I tried to analyze the number of repetitions needed to start to be able to distinguish between the two cases with and without the extra multiplication for a big key size, by plotting the minimum decryption time obtained for both cases with different numbers of repetitions of timing measurements in Figure 3. Each measurement is repeated 10 times to try to reduce the impact of noise, allowing to better visualize if the two cases are distinguishable or not.

The results show that for a large key, due to the variance in timing measurements, it is difficult to distinguish between the two cases—with and without the additional multiplication. In fact, for a small number of repetitions of the time measurements, the minimum decryption time obtained for the two cases can be reversed, meaning that the case with the additional multiplication (h1) may have a minimum decryption time lower than that of the case without additional multiplication (h0), as is the case for 100 repetitions.

So to have a good chance to distinguish between the two cases, it is necessary to perform a large number of repetitions of timing measurements. For my purpose, I chose to perform at least 500 repetitions of timing measurements for each message to have a good balance between the time taken for the attack and the chances of success of the attack.

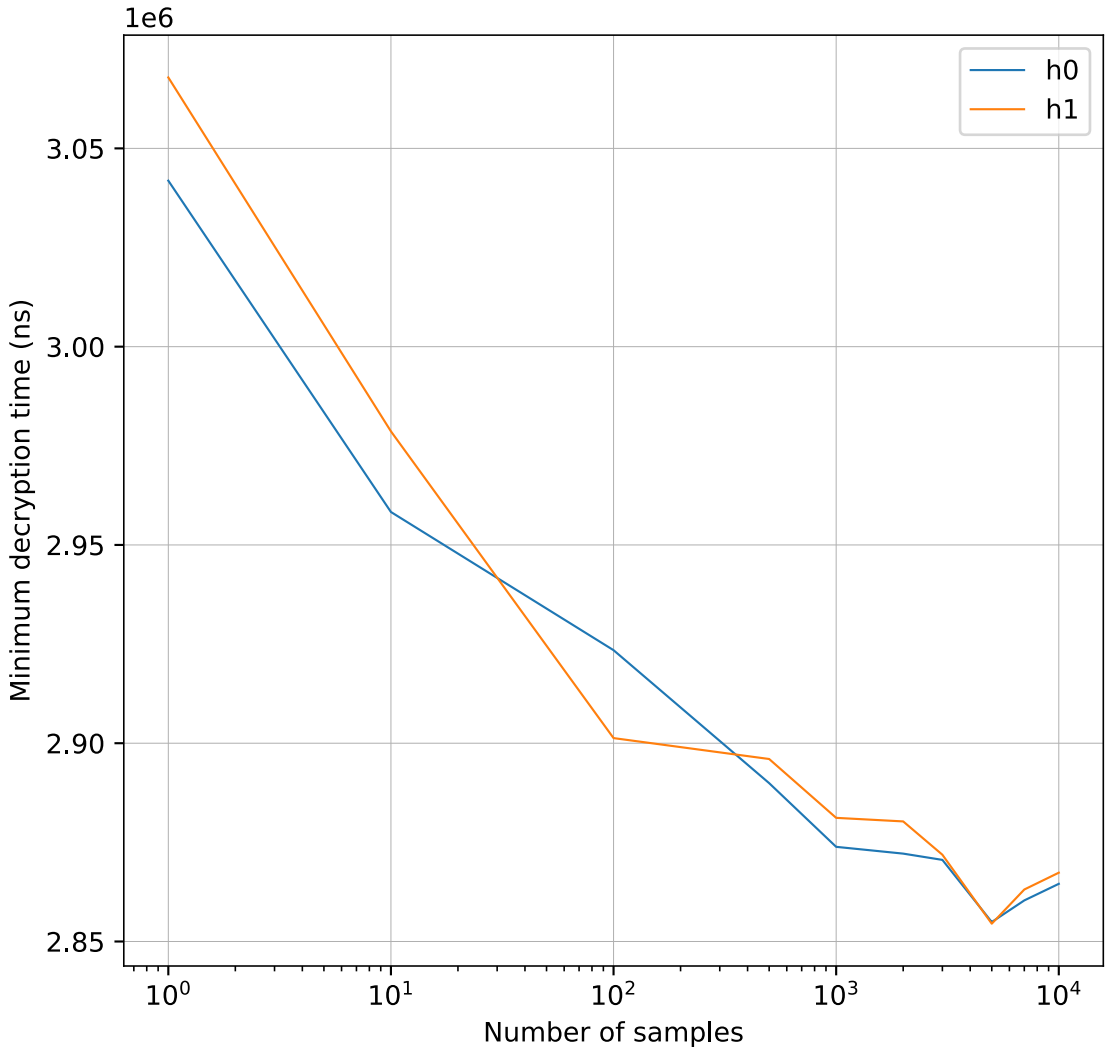


Figure 3: Convergence of minimum decryption time with number of samples

3.4.3 Performance of the attack

I first tried to implement the attack in Python with a simple version that does not look after noise, but it was not successful at all in recovering the private key due to the high noise caused by the Python interpreter and its optimizations. Indeed, the error rate was at best around 50%, meaning that the attack was making nothing else than random guesses for the bits of the private key d.

I then developed a version that uses repeated time measurements, a warm-up phase, and handles garbage collection, but it still hasn't managed to recover the private key. This was due to the fact that I was using the average of the timing measurements instead of the minimum. Note that I thought to remove the outliers from the timing measurements to try to reduce the impact of noise, but it was not sufficient to extract the signal from the noise.

So I tried using C++ and Rust to reduce the noise in the timing measurements, but that didn't allow me to recover the private key because the average was used again.

Finally, I implemented the attack in Python using the minimum as the estimator, which allowed me to significantly reduce the error rate of the attack and thus recover some bits of the private key d with a reasonable error rate.

I only managed to recover 5 bits of the private key d using 1000 messages and 10000 repetitions of timing measurements for each message. And I have assumed that the first 16 bits of the private key d were known. I use 3 candidates per iteration to try to reduce the impact of errors in the previous bits on the variance and thus reduce the number of errors in the recovered private key.

The attack took approximately 13 hours on a laptop with an AMD Ryzen 9 7950X3D processor.

```
Iteration 1/5: bit guessed = 1  (var h0 = 1.5563e+08, var h1 = 1.5628e+08)  ✗
WRONG

Iteration 2/5: bit guessed = 1  (var h0 = 1.5586e+08, var h1 = 1.5514e+08)  ✓
CORRECT

Iteration 3/5: bit guessed = 1  (var h0 = 1.5520e+08, var h1 = 1.5653e+08)  ✓
CORRECT

Iteration 4/5: bit guessed = 0  (var h0 = 1.5497e+08, var h1 = 1.5614e+08)  ✓
CORRECT

Iteration 5/5: bit guessed = 1  (var h0 = 1.5522e+08, var h1 = 1.5507e+08)  ✓
CORRECT


──────────────────────────────────────────
Candidate #1
  Key:        1234141
  Variance:   1.5475e+08
  Error rate: 20.00%  (1/5 bits wrong)
──────────────────────────────────────────
Candidate #2
  Key:        1496285
  Variance:   1.5507e+08
  Error rate: 0.00%  (0/5 bits wrong)
──────────────────────────────────────────
Candidate #3
  Key:        447709
  Variance:   1.5522e+08
  Error rate: 20.00%  (1/5 bits wrong)
──────────────────────────────────────────
```

Based on the results, we can see that one of the candidates is correct, while the other two candidates have an error rate of 20%, meaning they contain only one incorrect bit out of the five recovered bits, which is quite satisfactory given the noise present in the time measurements.

So the attack was successful in recovering some bits of the private key d with a reasonable error rate.

Given the computational power required for this attack, I did not attempt to recover additional bits or carry out the attack on the last bits of the private key d, which are more difficult to recover due to the greater noise present in the timing measurements.

4 Pearson-based timing attack

To make more effective use of the variations in time within the square-and-multiply algorithm, Pearson's correlation coefficient can be used instead of variance to determine which estimate of the private key bit d is most likely to be correct.

4.1 Pearson correlation coefficient

The Pearson correlation coefficient is a measure of the linear correlation between two variables, in this case, the timing measurements and the expected timings based on the guess for the bit of the private key d.

The Pearson correlation coefficient can be formally defined as:

$$r = \frac{\text{Cov}(T, t(C, x_b))}{\sigma_T \sigma_{t(C,x_b)}}$$

with:

- r the Pearson correlation coefficient.
- C the set of ciphertexts $C = \{c_1, c_2, ..., c_n\}$.
- T the vector of timing measurements obtained from the server for the set of ciphertexts C.
- $t(C, x_b)$ the vector of expected timings based on the guess x_b for the bit of the private key d for the set of ciphertexts C.
- σ_T the standard deviation of the timing measurements T.
- $\sigma_{t(C,x_b)}$ the standard deviation of the expected timings $t(C, x_b)$.

A value close to 1 indicates a strong positive correlation, meaning that the timing measurements and the expected timings are closely related. A value close to -1 indicates a strong negative correlation, and a value close to 0 indicates no linear correlation.

In fact, we expect the obtained time measurements to follow the variations predicted by the expected times, based on the assumption of the private key. This means that if the assumption is correct, we should observe a strong positive correlation between the time measurements and the expected times since the expected times explain the variations in the time measurements. On the other hand, if the assumption is incorrect, we should observe a weak correlation between the time measurements and the expected times since the expected times do not explain the variations in the time measurements.

4.2 Designing the model of expected timings

The model of expected timings is a crucial part of the Pearson-based approach, as it allows to estimate the expected timings based on the guess for the bit of the private key d and the set of ciphertexts C.

Indeed, the model must take into account the size of the ciphertexts that lead to different timings for the decryption operation, as well as the guess for the bit of the private key d that leads to different execution paths in the square-and-multiply algorithm and thus different timings.

A simple approach can consist to use the Hamming weight of the intermediate values in the square-and-multiply algorithm as a model for the expected timings.

The Hamming weight of a binary number is the number of bits that are set to 1 in the binary representation of the number. It can be used to estimate the complexity of the operations, as the operations that involve more bits set to 1 can be more complex for the hardware and thus take more time to execute. So for instance, an operation on 11111111 will take more time than on 10000000.

In my case, using length rather than Hamming distance significantly reduces the efficacy of the attack, but it might be interesting to try using other models to estimate the durations.

The cost computation for the square-and-multiply algorithm can be modeled as follows:

```
def square_and_multiply_with_cost(y, x, n):
    """
    Square-and-multiply algorithm with cost modeling.
    """
    s = 1
    y %= n
    cost = 0

    while x > 0:
        if x & 1:
            s = (s * y) % n
            cost += s.bit_count()
        y = (y * y) % n
        x >>= 1
    return s, cost
```

4.3 Advantages of the Pearson-based approach

Compared to the variance based approach, the Pearson correlation coefficient can be more efficient to determine which guess for the bit of the private key d is more likely to be correct, as it takes into account the relationship between the timing measurements and the expected timings based on the guess for the bit of the private key d, rather than just looking at the variance.

Indeed, Pearson based approach aims to explain the variations in the execution time of the square-and-multiply algorithm using a simple model that is independent of noise, machine architecture, algorithm implementation, the chosen programming language, and so on.

In this way, the attack depends only on the ability to capture the timing variations from a server.

On top of that, the Pearson approach allows to avoid performing a large number of timing measurements for each guess, allowing to significantly reduce the time taken for the attack.

For example, instead of performing 1,000 messages $\times$ 10,000 iterations $=$ 10 million time measurements for each hypothesis regarding the bit of the private key d, it is sufficient to calculate the expected cost for each ciphertext, which does not require repeated measurements and can be done in parallel. Next, the Pearson correlation coefficient between two

vectors of size 1,000 must be calculated, which can be done in a reasonable amount of time even for a large key.

4.4 Disadvantages of the Pearson-based approach

The main disadvantage of the Pearson-based approach is that it relies on the design of a good model for the expected timings based on the guess for the bit of the private key d.

If the model is not well designed, it can lead to a low Pearson correlation coefficient even for the correct guess for the bit of the private key d, making it difficult to distinguish between the correct and incorrect guesses.

And if the model is too complex, calculating the expected durations can take a long time, which can prolong the duration of the attack.

4.5 Results of the Pearson-based approach

Instead of taking 1000 messages and 10000 repetitions of timing measurements for each message, I took 3000 messages and 500 repetitions of timing measurements for each message, which allowed to significantly reduce the time taken for the attack to around 1h30.

And instead of recovering 5 bits of the private key d, I tried to recover 10 bits of the private key d.

To show you the importance of the estimator used for the timing measurements, I first tried to use the median as the estimator for the timing measurements, and then the minimum as the estimator for the timing measurements, and I compared the results of the two approaches.

The obtained results using the median are the following:

```
Iteration  1/10: bit guessed = 0  (r h0 = -0.0022, r h1 = -0.0104)  ✓ CORRECT
Iteration  2/10: bit guessed = 1  (r h0 = -0.0022, r h1 = -0.0018)  ✓ CORRECT
Iteration  3/10: bit guessed = 1  (r h0 = -0.0018, r h1 = +0.0014)  ✓ CORRECT
Iteration  4/10: bit guessed = 1  (r h0 = +0.0014, r h1 = +0.0022)  ✗ WRONG
Iteration  5/10: bit guessed = 1  (r h0 = +0.0022, r h1 = +0.0107)  ✓ CORRECT
Iteration  6/10: bit guessed = 0  (r h0 = +0.0107, r h1 = +0.0090)  ✓ CORRECT
Iteration  7/10: bit guessed = 0  (r h0 = +0.0107, r h1 = +0.0025)  ✗ WRONG
Iteration  8/10: bit guessed = 1  (r h0 = +0.0107, r h1 = +0.0136)  ✓ CORRECT
Iteration  9/10: bit guessed = 1  (r h0 = +0.0107, r h1 = +0.0145)  ✗ WRONG
Iteration 10/10: bit guessed = 1  (r h0 = +0.0118, r h1 = +0.0178)  ✓ CORRECT


──────────────────────────────────────────
Candidate #1
  Key:        60740829
  Pearson r:  +0.0178
  Error rate: 30.00%  (3/10 bits wrong)
──────────────────────────────────────────
Candidate #2
  Key:        52352221
  Pearson r:  +0.0162
  Error rate: 40.00%  (4/10 bits wrong)
──────────────────────────────────────────
Candidate #3
```

```
  Key:        18797789
  Pearson r:  +0.0145
  Error rate: 50.00%  (5/10 bits wrong)
──────────────────────────────────────────
```

The obtained results are not so bad as the error rate is around 30% for the best candidate, which is better than random guessing.

The results using the minimum as the estimator for the timing measurements are the following:

```
Iteration  1/10: bit guessed = 1  (r h0 = -0.0191, r h1 = -0.0121)  ✗ WRONG
Iteration  2/10: bit guessed = 0  (r h0 = -0.0121, r h1 = -0.0123)  ✗ WRONG
Iteration  3/10: bit guessed = 1  (r h0 = -0.0121, r h1 = -0.0077)  ✓ CORRECT
Iteration  4/10: bit guessed = 1  (r h0 = -0.0121, r h1 = -0.0036)  ✗ WRONG
Iteration  5/10: bit guessed = 1  (r h0 = -0.0062, r h1 = +0.0006)  ✓ CORRECT
Iteration  6/10: bit guessed = 0  (r h0 = +0.0006, r h1 = -0.0009)  ✓ CORRECT
Iteration  7/10: bit guessed = 1  (r h0 = +0.0006, r h1 = +0.0025)  ✓ CORRECT
Iteration  8/10: bit guessed = 1  (r h0 = +0.0025, r h1 = +0.0029)  ✓ CORRECT
Iteration  9/10: bit guessed = 1  (r h0 = +0.0029, r h1 = +0.0041)  ✗ WRONG
Iteration 10/10: bit guessed = 1  (r h0 = +0.0029, r h1 = +0.0103)  ✓ CORRECT

──────────────────────────────────────────
Candidate #1
  Key:        48157917
  Pearson r:  +0.0103
  Error rate: 10.00%  (1/10 bits wrong)
──────────────────────────────────────────
Candidate #2
  Key:        64935133
  Pearson r:  +0.0058
  Error rate: 20.00%  (2/10 bits wrong)
──────────────────────────────────────────
Candidate #3
  Key:        31380701
  Pearson r:  +0.0041
  Error rate: 30.00%  (3/10 bits wrong)
──────────────────────────────────────────
```

The obtained results are much better than the results obtained using the median as the estimator for the timing measurements, as the error rate is only 10% for the best candidate, which is significantly better than random guessing.

And compared to the variance-based approach, the attack was achieved in around 1h30, which is significantly faster than the 13 hours taken for the variance-based approach.

However, we can constate that the Pearson correlation coefficient (Pearson r) is very close to 0 for all candidates, even for the correct candidate, which can be explained by the fact that the timing variations are very small and thus the model for the expected timings is not enough precise to explain the small variations in the timing measurements.

Depending on the key size, the attack can be more or less effective, as for a large key size, the timing variations are smaller and thus it can be more difficult to distinguish between the correct and incorrect guesses based on the Pearson correlation coefficient. However the attack is still effective for a large key size, as it allows for instance to have an error rate of 30% for a key size of 500 bits, which is significantly better than random guessing.

To improve the Pearson-based approach, it would be interesting to try to design a better model for the expected timings, which can help to increase the Pearson correlation coefficient for the correct guess and thus make it easier to distinguish between the correct and incorrect guesses. Or it would be interesting to try to collect more timing measurements from the server to have more chance to capture the true timing variations.

4.6 Why the timing attack on RSA is still relevant

In general, servers are set up one time for many years, and they are optimized as much as the actual modern CPU architecture allows.

On top of that, the implementations on a server are often done in low-level programming languages such as C, which can allow to reduce the noise in the timing measurements and thus increase the chances of success of the attack compared to high-level programming languages such as Python.

In this way, the timing variations caused by the square-and-multiply algorithm running on a server can be more significant and thus easier to exploit for the attack compared to the timing variations caused by the square-and-multiply algorithm running on a local machine with a high optimized setup.

And even if the timing attack on RSA is not as simple as one might expect, it is still relevant to study it and try to implement it as it allows to understand the vulnerabilities of RSA and the importance of implementing countermeasures against timing attacks, such as designing the implementation to be constant-time. An implementation is constant-time if the execution time of the algorithm does not depend on the input values, which can help to mitigate the risk of timing attacks.

It is also important to understand that, even though the algorithm is theoretically resistant to all attacks, its implementation may still be vulnerable to side-channel attacks, such as timing attacks.

5 Fermat's factorization method on RSA

5.1 Security of RSA and factoring large integers

The private key d in RSA is computed based on the prime factors p and q of the modulus n. So another way to recover the private key d is to factor the modulus n into its prime factors p and q, and then compute d using the formula $d \equiv e^{-1} \bmod \varphi(n)$ with $\varphi(n) = (p-1)(q-1)$ the Euler's totient function. The inverse of e modulo $\varphi(n)$ can be computed using the Extended Euclidean Algorithm, which is an efficient method for computing the greatest common divisor of two integers and their multiplicative inverse.

But factoring large integers is a computationally hard problem, and the security of RSA relies on this fact.

However, if p and q are close to each other, meaning that the difference between p and q is small, then it becomes easy to factor n using Fermat's factorization method, as described by Hanno Böck in his paper "Fermat Factorization in the Wild" [3].

This paper is quite recent (2023) and shows that there are still some implementations of RSA that are vulnerable to Fermat's factorization method due to the key generation process that allows to generate prime numbers that are close to each other, which can lead to a significant security vulnerability for RSA.

That's why it is important to study Fermat's factorization method and understand how it works, as it can help to identify potential vulnerabilities in RSA implementations and to design countermeasures against this type of attack.

5.2 Description of Fermat's factorization method

Fermat's factorization method relies on the fact that n is a product of two odd primes p and q. As p and q are odd, there always exist an integer number which is at the middle of p and q, which is $x = \frac{p+q}{2}$. Then, we consider y as the distance between x and p (or q):

$$y = x - p = q - x$$

So we have:

- $p = x - y$
- $q = x + y$

If we express n in terms of x and y, we get:

$$n = p \times q = (x-y)(x+y) = x^2 - y^2$$

From this equation, we can deduce that $y^2 = x^2 - n$.

5.3 From theory to the algorithm

The Fermat's factorization method consists in finding the smallest integer x such that $y^2 = x^2 - n$ is a perfect square, meaning that y is an integer.

Once we find such an integer x, we can compute y as $y = \sqrt{x^2 - n}$, and then we can obtain the prime factors p and q as described in Section 5.2.

So the algorithm can be summarized as follows:

1. Compute $x = \lceil \sqrt{n} \rceil$.
2. Compute $y^2 = x^2 - n$.
3. If y^2 is a perfect square, then $y = \sqrt{y^2}$ and we can compute $p = x - y$ and $q = x + y$.
4. Otherwise, increment x and repeat from step 2.

5.4 Results of Fermat's factorization method on RSA

To test the effectiveness of Fermat's factorization method on RSA, I implemented the algorithm in Python and applied it to a modulus n that is a product of two close prime numbers.

I tried different gaps between the two prime numbers to see how it affects the method's performance, and I measured the time taken for the factorization to succeed or fail within a maximum number of iterations set to 2,000,000.

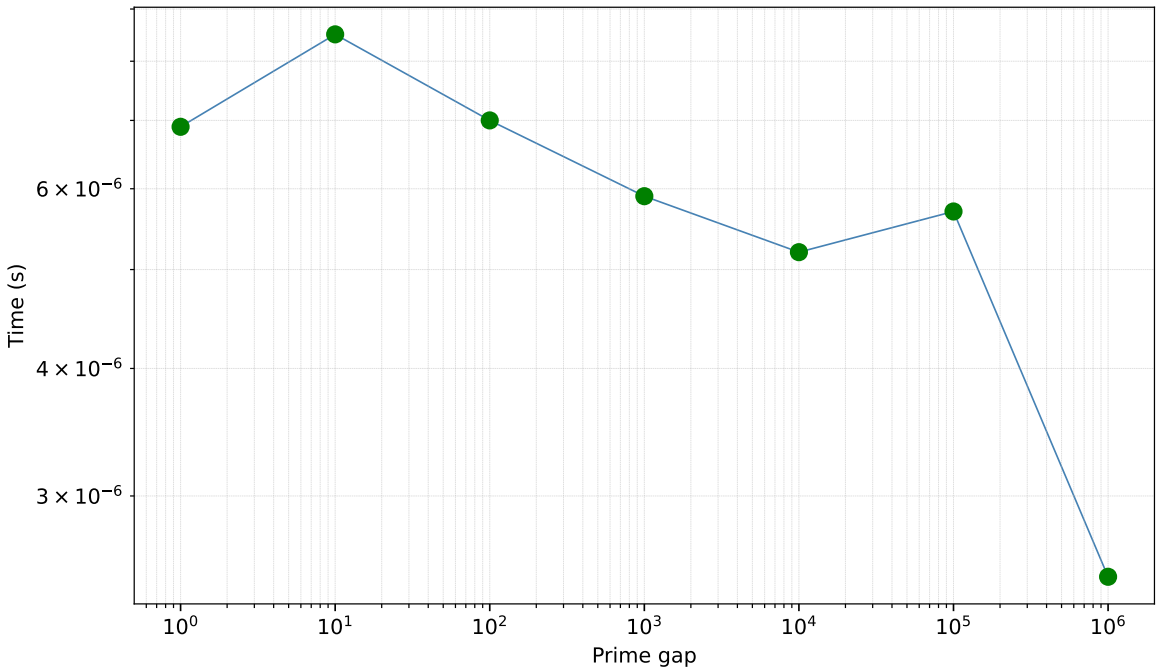


Figure 4: Fermat factorization with n: 2048 bit

On the Figure 4, all green points indicate that the factorization was successful within the maximum number of iterations, while red points indicate that the factorization was not successful within the maximum number of iterations.

We can constate that for all gaps up to 1,000,000, the factorization was successful within the maximum number of iterations in a very small time (less than 1 second), which shows that Fermat's factorization method can be very effective for factoring the modulus n when the prime factors p and q are close to each other.

Note that according to the primary number theorem, the number of prime numbers less than a given number x is approximately $\frac{x}{\log(x)}$. So the density of prime numbers can be estimated

as $\frac{x}{\log(x)} \cdot x$, which means that the average gap between two prime numbers around x is approximately $\log(x)$.

In the case of RSA with prime numbers of 1024 bits, the average gap between two prime numbers is approximately $\log(2^{1024}) = 1024 \log(2) \approx 710$. So the maximum gap tested in the graph is $1,000,000 \cdot 710 = 710,000,000 \approx 2^{29}$.

So if the prime numbers used have only the last 30 bits that differ, then the factorization can be done in a very short time using Fermat's factorization method.

Given the time required to find the two prime numbers, which involves checking whether a large number is prime each time, I do not test for gaps greater than 1,000,000, since the code already took three hours to compute the entire graph.

But it will be interesting to test the limit of the gap for which Fermat's factorization method is still effective, as it can help to determine the minimum gap that should be used for the prime numbers in RSA key generation to avoid this vulnerability.

The closeness of the prime factors p and q can be a significant vulnerability for RSA, as it can allow an attacker to factor the modulus n and thus recover the private key d in a very short time.

So it is crucial to ensure that the prime numbers used for RSA key generation are not too close to each other, as it can lead to a significant vulnerability for RSA.

6 Conclusion

In this article, we have explored two different approaches to attack RSA: a variance-based timing attack and a Pearson-based timing attack. The two approaches rely on the same principle of exploiting the timing variations caused by the square-and-multiply algorithm used for RSA decryption, but they differ in the way they analyze the timing measurements to determine which guess for the bit of the private key d is more likely to be correct.

The variance-based approach relies on the idea that the timing measurements for the case with the extra multiplication (h1) will have a higher variance than the case without the extra multiplication (h0), due to the fact that the extra multiplication can lead to more variations in the execution time of the square-and-multiply algorithm.

On the other hand, the Pearson-based approach relies on the idea that the timing measurements for the case with the extra multiplication (h1) will have a stronger positive correlation with the expected timings based on the guess for the bit of the private key d than the case without the extra multiplication (h0), due to the fact that the expected timings can explain better the variations in the timing measurements for the correct guess.

The results of the two approaches show that the Pearson-based approach can be more effective in recovering bits of the private key d with a lower error rate and in a shorter time compared to the variance-based approach, as it takes into account the relationship between the timing measurements and the expected timings based on the guess for the bit of the private key d.

However, the Pearson-based approach relies on the design of a good model for the expected timings, which can be a challenging task.

Finally, we have also explored Fermat's factorization method on RSA, which can be effective in factoring the modulus n when the prime factors p and q are close to each other, leading to a significant vulnerability for RSA.

6.1 Usage of AI

6.1.1 ChatGPT (Free)

- Research of articles and papers on timing attacks.
- Suggestions for timing attack implementations and optimizations.
- Suggestions for the design of the expected timings model for the Pearson-based approach.
- Summarization of some articles and papers on timing attacks.
- Creation of bibtex entries for the bibliography.

6.1.2 Claude Sonnet 4.6 (Free)

- Used initially for implementing the attack in Python, but it was finally done manually for more clarity.
- Improvement of some functions for the timing attack implementations in Python.
- Code review and suggestions for optimizations for the timing attack implementations.
- Suggestions for the design of the expected timings model for the Pearson-based approach.
- Summarization of some articles and papers on timing attacks.

- When tested, implementation of the attack in C++ and Rust.

6.1.3 Github Copilot (Student Plan)

- Auto-completion of code for the timing attack implementations in Python.
- Auto-completion of report writing.

6.1.4 DeepL (Free)

- Improvement of some sentences in the report to make them more clear and concise.

Bibliography

[1] P. C. Kocher, “Timing attacks on implementations of Diffie-Hellman, RSA, DSS, and other systems,” in *Annual international cryptology conference*, 1996, pp. 104–113.

[2] M. Ochoa, “Kocher's Timing Attack: A Journey from Theory to Practice.” Accessed: May 10, 2026. [Online]. Available: https://blog.zksecurity.xyz/posts/timing-kocher/

[3] H. Böck, “Fermat Factorization in the Wild.” [Online]. Available: https://eprint.iacr.org/2023/026